Projeto Final – Entrega da Fase 1: Modelagem Conceitual e Padrões de Sincronização

---

Aluno: Erik Rebelo da Silva

Sistema de Estudo: Kontado (Gestão de Despesas e Manutenção Veicular)

Tema: Especificação Arquitetural e Simulação Experimental de Mecanismos de Sincronização *Offline-First* para o Sistema Kontado.

1. Contextualização e Escopo da Fase 1

O sistema Kontado foi concebido para auxiliar motoristas no controle financeiro e na manutenção preditiva de veículos. Em cenários reais de deslocamento em rodovias e regiões com cobertura de rede móvel intermitente, a dependência exclusiva de conectividade contínua inviabiliza o registro imediato de abastecimentos, despesas e viagens.

Esta **Fase 1** estabelece a especificação arquitetural e os contratos formais de sincronização baseados no código-fonte dos repositórios do projeto (aplicação móvel em React Native com SQLite/Drizzle e backend REST em Node.js com Express/Prisma/PostgreSQL). O foco desta etapa consiste em formalizar a arquitetura de componentes, a máquina de estados da fila local (Padrão *Outbox*), os contratos de dados em JSON (*Delta Sync*) e as regras de reconciliação de concorrência (*Last-Write-Wins* e integridade de domínio veicular).

2. Arquitetura e Diagrama de Componentes

Em conformidade com os critérios formais da modelagem UML, a arquitetura do sistema é consolidada em três módulos principais e um pacote de banco de dados central:

- **Aplicativo Kontado (React Native):** Módulo único cliente responsável pela interface do motorista e pelas regras de negócio locais de veículos, abastecimentos, despesas e viagens.



- **Armazenamento Local (SQLite / Fila Outbox):** Módulo de persistência embarcada que garante leitura e escrita com latência zero durante períodos *offline*, retendo mutações pendentes na fila de saída.
- **API de Sincronização (REST / Mockoon):** Módulo servidor encarregado de receber lotes de mutações (*deltas*), validar esquemas e executar a reconciliação de conflitos,.
- **BD (Banco Central PostgreSQL):** Pacote de persistência consolidada na nuvem responsável pelo armazenamento definitivo e controle de versão dos registros.

3. Formalização do Padrão Outbox e Estado de Sincronização (SyncStatus)

A partir da análise do repositório móvel, a retenção de operações realizadas sem conectividade utiliza metadados de estado acoplados às tabelas locais do SQLite.

3.1 Máquina de Estados do Registro Local

- **PENDING (Pendente):** Registro criado, atualizado ou removido localmente no SQLite enquanto o dispositivo está *offline*; aguarda janela de conectividade para despacho em lote.

- **SYNCED (Sincronizado):** Registro processado e confirmado pelo servidor central após retorno HTTP 200 OK, com devida associação ao identificador remoto (remote_id).
- **CONFLICT (Conflito):** Divergência detectada pelo validador do servidor (ex.: edição concorrente com carimbo temporal inferior ou tentativa de regressão do hodômetro).
- **DELETED_LOCAL (Exclusão Lógica):** Registro marcado localmente com deleted_at (*soft delete*, alinhado à migração add_deleted_at_field do Prisma), aguardando propagação da exclusão para o banco central.

3.2 Esquema de Metadados de Controle Offline nas Tabelas do SQLite

As entidades sincronizáveis do domínio (vehicles, fuelings, expenses, trips e notes) incorporam

os seguintes atributos de controle para viabilizar a varredura incremental pelo serviço de sincronização:

Atributo Local	**Tipo (SQLite)**	**Função no Mecanismo de Sincronização Offline-First**
id_local	TEXT (UUID)	Chave primária gerada localmente no dispositivo, permitindo criação imediata sem depender do PostgreSQL.
remote_id	TEXT / INT	Identificador oficial atribuído pelo PostgreSQL após a primeira sincronização bem-sucedida.
sync_status	TEXT (ENUM)	Estado atual do registro na fila local (PENDING, SYNCED, CONFLICT).
created_in_app	BOOLEAN	Indicador de auditoria de origem móvel (mapeado da migração add_created_in_app_column da API).
updated_at	DATETIME	Carimbo de tempo da última mutação no cliente para comparação na política *Last-Write-Wins*.
deleted_at	DATETIME	Carimbo de tempo para exclusão lógica, sem a quebra de integridade da sua referencia.

4. Contratos de Payload JSON (Sincronização Incremental / Delta Sync)

Com base nos módulos src/controllers/sync.controller.ts e src/validators/sync.validator.ts da API e no serviço core/sync/simplify.service.ts do aplicativo, o protocolo evita a transferência completa do banco de dados (*Full Sync*), transmitindo exclusivamente o lote de mutações com status PENDING.

4.1 Envio em Lote (Request: POST /api/sync)


```
{
  "device_id": "moto-edge-60-client",
  "last_sync_at": "2026-10-21T18:00:00Z",
  "batch_timestamp": "2026-10-21T22:15:00Z",
  "deltas": {
    "vehicles": [
      {
        "id_local": "veh-local-01",
        "remote_id": "veh-pg-99",
        "operation": "UPDATE",
        "updated_at": "2026-10-21T21:40:00Z",
        "data": {
          "mileage": 54320,
          "fuelTankVolume": 38.5
        }
      }
    ],
    "fuelings": [
      {
        "id_local": "fuel-local-101",
        "remote_id": null,
        "operation": "INSERT",
        "created_in_app": true,
        "updated_at": "2026-10-21T21:35:10Z",
        "data": {
          "vehicleId": "veh-pg-99",
          "liters": 40.0,
```

```
        "pricePerLiter": 5.89,
        "totalCost": 235.60,
        "odometer": 54300,
        "fuelTypeId": 1,
        "stationId": "st-tapajo-01"
      }
    }
  ],
  "expenses": [
    {
      "id_local": "exp-local-204",
      "remote_id": null,
      "operation": "INSERT",
      "created_in_app": true,
      "updated_at": "2026-10-21T21:40:00Z",
      "data": {
        "vehicleId": "veh-pg-99",
        "categoryId": 3,
        "amount": 150.00,
        "description": "Alinhamento e balanceamento",
        "date": "2026-10-21T21:40:00Z",
        "odometer": 54320
      }
    }
  ]
 }
}
```


4.2 Exemplo ontrato de Resposta Consolidada do Servidor (Response: HTTP 200 OK)


```
{
 "status": "SYNC_COMPLETED",
 "processed_at": "2026-10-21T22:15:01Z",
 "mappings": [
  {
    "entity": "fuelings",
    "id_local": "fuel-local-101",
```

```
      "remote_id": "fuel-pg-884",
      "status": "SYNCED"
    },
    {
      "entity": "expenses",
      "id_local": "exp-local-204",
      "remote_id": "exp-pg-512",
      "status": "SYNCED"
    }
  ],
  "conflicts": []
}
```


5. Regras Formais de Reconciliação de Concorrência

A fim de garantir a consistência eventual entre o armazenamento local SQLite e o banco central PostgreSQL, temos o protocolo protocolo que adota três regras determinísticas que guiarão a bateria de ensaios experimentais na Fase 2:

1. **Resolução Temporal *Last-Write-Wins* (LWW):** Em operações de atualização (UPDATE) sobre um mesmo registro remoto, a API compara o atributo updated_at enviado pelo cliente com o updated_at persistido no PostgreSQL. Se a marca temporal do cliente for maior ou igual à do servidor, a atualização é aplicada; caso seja anterior, a escrita é rejeitada e o estado consolidado do servidor prevalece.
2. **Idempotência de Inserção (Mapeamento id_local → remote_id):** Para evitar duplicação de registros financeiros em caso de queda de conexão durante o recebimento da confirmação (*timeout* após escrita no banco), o servidor valida a chave composta (device_id, id_local). Se o evento já tiver sido processado anteriormente, a API apenas reenvia o mapeamento do remote_id existente sem duplicar a linha no PostgreSQL.
3. **Integridade de Domínio Veicular (Monotonicidade de Quilometragem):** Em alinhamento com o histórico de quilometragem da API (vehicle_mileage_snapshot), o hodômetro do veículo não admite regressão temporal indevida. Caso um lote *offline* atrasado tente atualizar a quilometragem atual do veículo para um valor inferior ao já consolidado no servidor, a redução do hodômetro é bloqueada com alerta de conflito (HTTP 409 / status: CONFLICT), preservando-se contudo o registro contábil da despesa ou abastecimento associado.